

Extensión de la Práctica Principal de Procesadores de Lenguajes (Examen febrero 2021)

Esta práctica consiste en la implementación mediante JFlex y Cup de un compilador de un pequeño lenguaje de programación, similar a C, denominado lenguaje PLX. El lenguaje PLX-2021 es una extensión del lenguaje PLX que se describe como prácticas de la asignatura, pero en esta versión extendida se requieren algunas funciones adicionales. Se presupone que todos los elementos del lenguaje PLX anteriores están presentes en el lenguaje PLX-2021 y que no se modifica su funcionamiento al incluir los nuevos elementos de esta extensión, aunque obviamente en este examen solo se evaluarán los casos que se describen en este enunciado, minimizando en la medida de lo posible el uso de funcionalidades anteriores. En concreto, para este ejercicio se utilizarán explícitamente solo los tipos “int”, “float” y “char”.

EL CÓDIGO FUENTE (Lenguaje PLX):

El lenguaje PLX-2021 incluye todas las sentencias de control definidas en el lenguaje PLX anterior y alguna más. El lenguaje intermedio CTD, no es necesario modificarlo.

ASPECTOS LEXICOS

Cualquier duda que surja al implementar, y que no estuviera suficientemente clara en este enunciado debe resolverse de acuerdo a las especificaciones del lenguaje JAVA o bien el lenguaje C .

COMPILADOR DE PRUEBA

Se proporciona una versión compilada del compilador, que puede usarse como referencia para la generación de código. No es necesario que el código generado por el compilador del alumno, sea exactamente igual al generado por el compilador de prueba, basta con que produzca los mismos resultados al ejecutarse para todas las entradas.

El compilador de prueba se entrega solamente a título orientativo. Si hubiese errores en el compilador de prueba, prevalecen las especificaciones escritas en este enunciado. Estos posibles errores en ningún caso eximen al alumno de realizar una implementación correcta.

NOTA:

La versión compilada de “plx” se ha actualizado en el repositorio para cubrir los cambios requeridos en este ejercicio, por lo que aunque ya se hubiera descargado anteriormente debería actualizarse.

Operadores mas unario.

Se permite el uso del operador unario + que afecta a la expresión situada a su derecha. Este operador no modifica el valor, es simplemente una opción sintáctica, al igual que el operador unario menos. Solo se puede aplicar a expresiones de tipo numérico, enteros y reales. Si se aplica a un carácter, lo convierte implícitamente en un numero entero.

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>int x; x = 3 * -2 + +5 * 6; print(x);</pre>	<pre>\$t0 = -2; \$t1 = 3 * \$t0; \$t2 = 5 * 6; \$t3 = \$t1 + \$t2; x = \$t3; print x;</pre>	24
<pre>int numero; numero = - +4 -+ -+-3; print(numero);</pre>	<pre>\$t0 = -4; \$t1 = -3; \$t2 = -\$t1; \$t3 = \$t0 - \$t2; numero = \$t3; print numero;</pre>	-7
<pre>char ch; ch = +'A'; print(ch);</pre>	<pre># Error de tipos error;</pre>	
<pre>int num; num = + 'A' + +'+' + 67; print(num);</pre>	<pre>\$t0 = 65 + 43; \$t1 = \$t0 + 67; num = \$t1; print num;</pre>	175
<pre>int x; int y; x = +3 * +5 + + -2; y = 3 * 5 -2; print(x*y);</pre>	<pre>\$t0 = 3 * 5; \$t1 = -2; \$t2 = \$t0 + \$t1; x = \$t2; \$t3 = 3 * 5; \$t4 = \$t3 - 2; y = \$t4; \$t5 = x * y; print \$t5;</pre>	169

Sentencia *repeat*.

Implementar una sentencia *repeat*, parecida a la sentencia *for*, pero en la que solo se especifica un número de iteraciones. El valor de repetición viene dado por una expresión.

Al igual que en otras sentencias de control, como las sentencias *for* y *while*, etc. el número de iteraciones puede variar si durante la ejecución varía el valor de la expresión, es decir, la expresión se vuelve a evaluar cada vez que se realiza una iteración y el bucle no termina hasta que el número de iteraciones sobrepasa el valor de la expresión.

La sentencia *repeat* se puede anidar y combinar con cualquier otra sentencia de control.

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>int n; n=5; repeat(n) { print(n); }</pre>	<pre>n = 5; \$t0 = 1; L0: if (n < \$t0) goto L1; print n; \$t0 = \$t0 + 1; goto L0; L1:</pre>	<pre>5 5 5 5 5</pre>
<pre>int n; n=5; repeat(n) { print(n); n = n-1; }</pre>	<pre>n = 5; \$t0 = 1; L0: if (n < \$t0) goto L1; print n; \$t1 = n - 1; n = \$t1; \$t0 = \$t0 + 1; goto L0; L1:</pre>	<pre>5 4 3</pre>
<pre>int n; n=5; repeat(n) repeat(n) { print(n); n = n-1; } print(n);</pre>	<pre>...</pre>	<pre>5 4 3 2 1</pre>
<pre>int n; n=5; repeat (n) { n = n-1; repeat(n) print(n+1); }</pre>		<pre>5 5 5 5 4 4 4 3 3</pre>
<pre>int n; n=1; repeat (n++ + ++n) { print(--n); repeat(n--) print(n); }</pre>		<pre>2 1 1 0</pre>
<pre>int n; repeat (1+2*3-4) { n = n+5; print(0); repeat (6-n/2) print(n); }</pre>		<pre>0 5 5 5 5 0 10 0</pre>

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre> int n; n=10; int i; i=0; repeat(n) { print(n); i = i+1; if (i==2) { n=5; } } </pre>		10 10 5 5 5
<pre> int n; n=10; repeat(n) { print(n); n = n-1; if (n%2==0) { n = n/2; } } </pre>		10 9 4
<pre> int i; int z; i=10; repeat (i) { for(i=10; i>0; i--) z++; } print(z); for(i=10; i>0; i--) { repeat(i) z++; } print(z); </pre>		10 65
<pre> int i; int j; int z; i=1; j=1; for(i=1; i<10; i++) repeat (i) while(j<10) repeat ((i+j)/j) z = z + i + j++; print(z) </pre>		54

Sentencia *repeat just*.

Es una variante semántica de la sentencia anterior, en el que el número de iteraciones se fija al principio de la ejecución y no se vuelve a calcular aunque se modifique su valor durante la ejecución del cuerpo de la sentencia.

En el caso en el que la expresión sea constante, o no dependa de variables que se modifiquen en el cuerpo de la función la sentencia *repeat just* es exactamente igual que la sentencia *repeat*.

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>int n; n=5; repeat just (n) { print(n); n = n-1; }</pre>	<pre> n = 5; \$t1 = n; L0: \$t0 = \$t0 + 1; if (\$t1 < \$t0) goto L1; print n; \$t2 = n - 1; n = \$t2; goto L0; L1:</pre>	<pre>5 4 3 2 1</pre>
<pre>int n; n=3; int m; m=3; repeat just (n*m) { print(n*m); m = m-1; n = n-1; }</pre>	<pre>...</pre>	<pre>9 4 1 0 1 4 9 16 25</pre>
<pre>int n; n=3; int m; m=3; repeat just (n) { repeat just (m) { print(n*m); m = m-1; } n = n-1; }</pre>	<pre>;</pre>	<pre>9 6 3</pre>
<pre>int i; int z; i=10; repeat just (i) { for(i=10; i>0; i--) z++; } print(z); for(i=10; i>0; i--) { repeat just (i) z++; } print(z);</pre>		<pre>100 155</pre>
<pre>int n; n=5; int i; int z; repeat just (n) { print(n); i = i+1; repeat just (n-i) { repeat (i) z = z+i; n=n-1; } } print(z);</pre>		<pre>5 1 1 1 1 4</pre>

Introducción del tipo *struct*.

Añadir al lenguaje PLX la definición de estructuras similares a las que se definen en lenguaje C.. Los campos que no se hayan inicializado tienen por defecto el valor 0. Para acceder a cada uno de los campos se usa el operador punto . que tiene la mayor prioridad de todos los operadores.

Los campos de las estructuras pueden ser de tipo `int`, `float` o `char`, o bien otras estructuras. Las estructuras se pueden anidar.

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>struct { int a; } p; p.a = 1; print(p.a);</pre>	<pre>p\$a = 1; print p\$a;</pre>	1
<pre>struct { int a; float b; } p; p.a = 1; p.b = 2.5; print(p.a); print(p.b);</pre>	<pre>p\$a = 1; p\$b = 2.5; print p\$a; print p\$b;</pre>	1 2.500000
<pre>struct { int a; int b; int c; } p,q; p.a = 1+2*3; p.b = 2+3*4; q.a = 3*4+5; print(p.a); print(q.b); print(q.a);</pre>	<pre>p\$a = 1; p\$b = 2; q\$c = 3; print p\$a; print q\$b; print q\$c;</pre>	7 0 17
<pre>struct { int a; struct { int b; int c; } q; } p; p.a = 1; p.q.b = 2; p.q.c = 3; print(p.a); print(p.q.b); print(p.q.c);</pre>	<pre>p\$a = 1; p\$q\$b = 2; p\$q\$c = 3; print p\$a; print p\$q\$b; print p\$q\$c;</pre>	1 2 3
<pre>struct { struct { int n2; struct { int n1; int n2; } omega; } q; char ch; int n1; } p; p.n1 = 3; p.q.omega.n1 = p.n1; p.q.omega.n2 = p.q.omega.n1 / p.n1; p.ch = 'R'; print(p.ch); print(2*p.q.omega.n2); p.ch = 'D'; print(p.ch); print(2*p.n1-4);</pre>	<pre>p\$n1 = 3; p\$q\$omega\$n1 = p\$n1; \$t0 = p\$q\$omega\$n1 / p\$n1; p\$q\$omega\$n2 = \$t0; p\$ch = 82; printc p\$ch; \$t1 = 2 * p\$q\$omega\$n2; print \$t1; p\$ch = 68; printc p\$ch; \$t2 = 2 * p\$n1; \$t3 = \$t2 - 4; print \$t3;</pre>	R 2 D 2

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre> struct { struct { int a,b; int c,d,e,f; int g; } p; int x,y,z; } alfa,beta; alfa.x = 5; alfa.y = 4; alfa.z = 3; beta.x = 2; beta.p.e = 1; beta.z = 6; print(alfa.x+2*beta.p.e+3*alfa.z); </pre>	<pre> alfa\$x = 5; alfa\$y = 4; alfa\$z = 3; beta\$x = 2; beta\$p\$e = 1; beta\$z = 6; \$t0 = 2 * beta\$p\$e; \$t1 = alfa\$x + \$t0; \$t2 = 3 * alfa\$z; \$t3 = \$t1 + \$t2; print \$t3; </pre>	16
<pre> struct { struct {int x;} x; struct {int y;} y; int z; } alfa; alfa.x.x = 3; alfa.y.y = 4; print(alfa.x.x+2*alfa.y.y); </pre>	<pre> alfa\$x\$x = 3; alfa\$y\$y = 4; \$t0 = 2 * alfa\$y\$y; \$t1 = alfa\$x\$x + \$t0; print \$t1; </pre>	11
<pre> struct { struct {int x;} a; struct {int y;} b; int a; } alfa; alfa.a.x = 3; alfa.b.y = 4; print(alfa.a.x+2*alfa.b.y); </pre>	<pre> #ERROR: Camppo duplicado error; halt; </pre>	
<pre> struct { struct {int x;} a; struct {int y;} b; int c; } alfa; alfa.a.x = 3; alfa.a.y = 4; print(alfa.a.x+2*alfa.b.y); </pre>	<pre> # ERROR: Campo no encontrado error; halt; </pre>	
<pre> struct { int a; struct { } b; int c; } d; </pre>	<pre> # syntax error error; halt; </pre>	

Copia de variables de tipo *struct*. NO PUNTUA PARA EL EXAMEN

Se dice que dos tipos *struct* son equivalentes, si tienen los mismos campos, de los mismos tipos y en el mismo orden, aunque los nombres de los campos no coincidan. Implementar la asignación entre variables de tipo *struct* con equivalencia estructural en

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre> struct { int a; int b; } p; struct { int a; int b; } q; p.a = 7; p.b = 2; q = p; print(q.a - 2*q.b); </pre>	<pre> p\$a = 7; p\$b = 2; q\$a = p\$a; q = p; q\$b = p\$b; \$t0 = 2 * q\$b; \$t1 = q\$a - \$t0; print \$t1; </pre>	3
<pre> struct { int a; int b; int c; } p; struct { int a; int b; int z } q; p.a = 7; p.b = 2; p.c = 9; q = p; print(q.a-2*q.b+3*q.z); </pre>	<pre> p\$a = 7; p\$b = 2; p\$c = 9; q\$a = p\$a; q = p; q\$b = p\$b; q\$z = p\$c; \$t0 = 2 * q\$b; \$t1 = q\$a - \$t0; \$t2 = 3 * q\$z; \$t3 = \$t1 + \$t2; print \$t3; </pre>	30
<pre> struct { int a; struct { struct { int b; } x; } y; struct { int c; int d; } z; } p; struct { int a; struct { struct { int bb; } xx; } yy; struct { int c; int d; } zz; } q; p.a = 2; p.y.x.b = 3; p.z.c = 4; p.z.d = 5; q=p; print(q.a); print(q.yy.xx.bb); print(q.z.z.c); print(q.z.z.d); </pre>	<pre> p\$a = 2; p\$y\$x\$b = 3; p\$z\$c = 4; p\$z\$d = 5; q\$a = p\$a; q = p; q\$yy\$xx\$bb = p\$y\$x\$b; q\$yy\$xx = p\$y\$x; q\$yy = p\$y; q\$zz\$c = p\$z\$c; q\$zz = p\$z; q\$zz\$d = p\$z\$d; print q\$a; print q\$yy\$xx\$bb; </pre>	2 3 4 5

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre> struct { struct { int a; int b; } y; struct { int c; int d; } z; } p; struct { int a; int b; } x; x.a = 2; x.b = 3; p.y = x; p.z = x; print(p.y.a); print(p.y.b); print(p.z.c); print(p.z.d); </pre>	<pre> x\$a = 2; x\$b = 3; p\$y\$a = x\$a; p\$y = x; p\$y\$b = x\$b; p\$z\$c = x\$a; p\$z = x; p\$z\$d = x\$b; print p\$y\$a; print p\$y\$b; print p\$z\$c; print p\$z\$d; </pre>	<pre> 2 3 2 3 </pre>
<pre> struct { struct { int a; int b; } y; struct { int c; int d; } z; } p; struct { int a; int b; } x; x.a = 2; x.b = 3; p.y = x; p.z = x; p.y.a = p.y.a+2; p.y.b = p.y.b+2; x = p.y; print(p.z.c); print(p.z.d); print(x.a); print(x.b); </pre>	<pre> x\$a = 2; x\$b = 3; p\$y\$a = x\$a; p\$y = x; p\$y\$b = x\$b; p\$z\$c = x\$a; p\$z = x; p\$z\$d = x\$b; \$t0 = p\$y\$a + 2; p\$y\$a = \$t0; \$t1 = p\$y\$b + 2; p\$y\$b = \$t1; x\$a = p\$y\$a; x = p\$y; x\$b = p\$y\$b; print p\$z\$c; print p\$z\$d; print x\$a; print x\$b; </pre>	<pre> 2 3 4 5 </pre>

EL CÓDIGO OBJETO (Código de tres direcciones):

El código objeto CTD implementa una maquina abstracta con infinitos registros a los que se accede mediante variables. Todas las variables se considera que están previamente definidas y que su valor inicial es 0. No se diferencia entre números enteros y reales, salvo para realizar operaciones.

El conjunto de instrucciones del código intermedio, y su semántica son las siguientes:

Instrucción	Acción
<code>x = a ;</code>	Asigna el valor de a en la variable x
<code>x = a + b ;</code>	Suma los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a - b ;</code>	Resta los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a * b ;</code>	Multiplica los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a / b ;</code>	Divide (div. entera) los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a +r b ;</code>	Suma de dos valores reales
<code>x = a -r b ;</code>	Resta de dos valores reales
<code>x = a *r b ;</code>	Multiplicación de dos valores reales
<code>x = a /r b ;</code>	Division de dos valores reales
<code>x = (int) a ;</code>	Convierte un valor real a, en un valor entero, asignándose a la variable x
<code>x = (float) a ;</code>	Convierte un valor entero a, en un valor real, asignándose a la variable x
<code>x = y[a] ;</code>	Obtiene el a-esimo valor del array y, asignando el contenido en x
<code>x[a] = b ;</code>	Coloca el valor b en la a-esima posición del array x
<code>x = *y ;</code>	Asigna a x el valor contenido en la memoria referenciada por y.
<code>*x = y ;</code>	Asigna el valor y en la posición de memoria referenciada por x.
<code>x = &y ;</code>	Asigna a x la dirección de memoria en donde esta situado el obteto y.
<code>goto l ;</code>	Salto incondicional a la posición marcada con la sentencia "label l"
<code>if (a == b) goto l ;</code>	Salta a la posición marcada con la sentencia "label l", si y solo si el valor de a es igual que el valor de b
<code>if (a != b) goto l ;</code>	Salta a la posición marcada con la sentencia "label l", si y solo si el valor de a es distinto que el valor de b
<code>if (a < b) goto l ;</code>	Salta a la posición marcada con la sentencia "label l", si y solo si el valor de a es estrictamente menor que el valor de b.
<code>l:</code>	Indica una posición de salto.
<code>label l ;</code>	Indica una posición de salto. Es otra forma sintáctica equivalente a la anterior.
<code>function f :</code>	Indica una posición de salto para comienzo de una función.
<code>end f ;</code>	Indica el final del código de una función.
<code>param n = x;</code>	Indica que x debe usarse como parámetro n-simo en la llamada a la próxima función.
<code>x = param n ;</code>	Asigna a la variable x el valor del parámetro n-simo definido antes de la llamada a la función.
<code>call f ;</code>	Salto incondicional al comienzo de la función f. Al alcanzar la sentencia return el control vuelve a la instrucción inmediatamente siguiente a esta
<code>gosub l ;</code>	Salto incondicional a la etiqueta f. Al alcanzar la sentencia return el control vuelve a la instrucción inmediatamente siguiente a esta
<code>return ;</code>	Salta a la posición inmediatamente siguiente a la de la instrucción que hizo la llamada (call f) o (gosub l)
<code>write a ;</code>	Imprime el valor de a (ya sea entero o real)
<code>writec a ;</code>	Imprime el carácter Unicode correspondiente al número a
<code>print a ;</code>	Imprime el valor de a, y un salto de línea
<code>printc a ;</code>	Imprime el carácter Unicode correspondiente al número a, y un salto de línea
<code>error ;</code>	Indica una situación de error, pero no detiene la ejecución.
<code>halt ;</code>	Detiene la ejecución. Si no aparece esta instrucción la ejecución se detiene cuando se alcanza la última instrucción de la lista.
<code># ...</code>	Cualquier línea que comience con un # se considera un comentario.

En donde **a**,**b** representan tanto variables como constantes enteras, **x**,**y** representan siempre una variable, **n** representa un numero entero, **l** representa una etiqueta de salto y **f** un nombre de función.

IMPLEMENTACIÓN DE LA PRÁCTICA:

Se proporciona una solución compilada del ejercicio (versiones para Linux, Windows y Mac). Esto puede servir de ayuda para comprobar los casos de prueba y las instrucciones intermedio,. No es necesario que el código generado sea idéntico al que se propone como ejemplo (que de hecho no es óptimo), basta con que sea equivalente, es decir que dé los mismos resultados al ejecutar los casos de prueba. Para compilar y ejecutar un programa en lenguaje PLX, pueden utilizarse las instrucciones

	<i>Linux</i>
Compilación	<code>java PLXC prog.plx prog.ctd</code>
Ejecución	<code>./ctd prog.ctd</code>

En donde `prog.plx` contiene el código fuente en PLX, `prog.ctd` es un fichero de texto que contiene el código intermedio válido según las reglas gramaticales de este lenguaje. El programa `plx` es un *script* del *shell* del sistema operativo que llama a (`java PLXC`), que es el programa que se pide construir en este ejercicio. El programa `ctd` es un interprete del código intermedio. Asimismo, para mayor comodidad se proporciona otro *script del shell* denominado `plx` que compila y ejecuta en un solo paso, y al que se pasa el nombre del fichero sin extensión.

	<i>Linux</i>
Compilación + Ejecución	<code>./plx prog</code>

NOTAS IMPORTANTES:

1. Toda práctica debe contener al menos tres ficheros denominados “`PLXC.java`”, “`PLXC.flex`” y “`PLXC.cup`”, correspondientes respectivamente al programa principal y a las especificaciones en JFlex y Cup. Para realizar la compilación se utilizarán las siguientes instrucciones:

```
cup PLXC.cup
jflex PLXC.flex
javac *.java
```

y para compilar y ejecutar el programa en PLX

```
java PLXC prog.plx prog.ctd
./ctd prog.ctd
```

2. Puede ocurrir que al descargar los ficheros y descomprimirlos en Linux se haya perdido el carácter de fichero ejecutable. Para poder ejecutarlos debe modificar los permisos:

```
chmod +x plx plxc ctd
chmod +x plx-linux plxc-linux ctd-linux
```

3. El programa `ctd`, interprete del código intermedio, tiene una opción `-v` para generar trazas que pueden ayudar en la depuración de errores:

```
./ctd -v prog.ctd
```

4. Los ejemplos que se proponen como casos de prueba no definen exhaustivamente el lenguaje. Para implementar esta práctica es necesario generar otros casos de prueba de manera que se garantice un funcionamiento en todos los casos posibles, y no solo en este limitado banco de pruebas.
5. En todas las pruebas en donde el código **plx** produce un “*error*”, para comprobar que el compilador realmente detecta el error, se probará también que el código corregido compila adecuadamente, y si no es así la prueba no se considerará correcta.